

Permissions Demystified



Ever get a **"Permission Denied"** error in Linux? It's not random—it's actually a super clever security feature working exactly as designed. Here's the thing: **rwX** (read, write, execute) on three levels (user, group, other) gives you granular control. But most people just do **chmod 777** and call it a day. Don't be most people!

Prepared By: Hosni Zaaraoui

The Problem: Why Permissions Even Exist

Imagine a shared Linux server with 50 users. Ali's SSH keys are in his home directory. Bob's database credentials are in his config file. Without permissions, Bob could just...read Ali's keys. Game over.

Permissions are your security guardrail. They prevent chaos, data theft, and accidental (or malicious) damage.

The beauty? Linux doesn't force you to use complicated ACLs or Active Directory. It has a dead simple system that works everywhere: **rwX on user, group, and other.**



Linux Fundamentals



The Fundamentals: rwx Breakdown

Every file and directory in Linux has three permission types:

r — Read

- **For files:** Can you view the contents? (`cat file.txt`, `less readme.md`)
- **For directories:** Can you list what's inside? (`ls /home/user/`)
- Without read on a directory, you can't see what files exist there, even if you have write permission

w — Write

- **For files:** Can you modify or delete the contents?
- **For directories:** Can you create, delete, or rename files inside?
- This is the dangerous one. Write permission on a directory means you can delete files you don't own
- **Key insight:** You don't need write permission on a file to delete it—you need write on the directory containing it



Linux Fundamentals



x — Execute

- **For files:** Can you run this as a program? (./script.sh, /usr/bin/python3)
- **For directories:** Can you enter it and access files inside? (This is weird but crucial)
- A directory without execute permission? You can't cd into it, even if you have read permission
- This is the gotcha. Most people forget: **directories need x to be traversable**

```
hosni ~ 07:23 mkdir directory
hosni ~ 07:23 ls -la | grep directory
drwxrwxr-x - hosni 26 Nov 07:23 directory
hosni ~ 07:23 cd directory
hosni directory 07:23 ls -la
hosni directory 07:23 cd ..
hosni ~ 07:23 chmod u-x directory
hosni ~ 07:24 ls -la | grep directory
drw-rwxr-x - hosni 26 Nov 07:23 directory
hosni ~ 07:24 cd directory
cd: permission denied: directory
```

The Three Levels: User, Group, Other

Linux breaks permissions into three categories:

-**rw**x**rw**x**rw**x

| | +-- **Other** (everyone else on the system)

| +----- **Group** (users in a specific group)

+----- **User** (the file owner)

User (owner)

The person who created the file or explicitly owns it. Usually you. This person has the most control.

Group

A collection of users who share common permissions. Think of it as a team. The file can be assigned to any group on the system.

Common groups:

- **wheel** or **sudo** — administrators
- **docker** — can run Docker commands
- **www-data** — web server user
- Custom groups — developers, devops, etc.

Other (world)

Everyone else on the system. If they're not the owner and not in the group, they get these permissions.

Reading Permissions: Examples

```
-rw-r--r-- 1 ali developers 4096 Nov 24 10:30 report.pdf
```

Let's decode this:

Part	Meaning
-	Regular file (not a directory)
rw-	User (ali): read , write , execute
r--	Group (developers): read , write , execute
r--	Other: read , write , execute

Translation: Ali can **read** and **edit**. Developers can **only read**. Everyone else can **only read**.

Another example: Directory

```
drwxr-xr-x 2 root root 4096 Nov 24 10:30 /etc/ssh/
```

Part	Meaning
d	Directory
rwX	Owner (root): can read, write, delete files inside, and enter
r-x	Group (root): can see what's inside and enter, but not modify
r-x	Other: can see what's inside and enter, but not modify

Translation: Root **controls everything**. Everyone can **browse** and **enter**, but **not modify**. This is intentional—SSH configs need to stay locked down.



The chmod Cheat Sheet: What You Actually Need

chmod changes permissions. Two ways to use it:

Symbolic (Readable)

```
chmod u+rw file.txt      # User: add read + write
```

```
chmod g-w file.txt      # Group: remove write
```

```
chmod o-rwx file.txt    # Other: remove all permissions
```

```
chmod a+x script.sh     # All: add execute
```

```
chmod u=rwx,g=rx,o= file # User: rwx, Group: rx, Other: nothing
```



Octal (Faster)

Numbers represent permissions:

- $r = 4$
- $w = 2$
- $x = 1$

Add them up for each level:

`chmod 755 script.sh`

||+-- Other : $r+x = 5$ (read + execute)

|+--- Group : $r+x = 5$ (read + execute)

+----- User : $r+w+x = 7$ (all permissions)

Common patterns you should know:

Number	Meaning	Use case
755	<code>rwxr-xr-x</code>	Scripts, executables, public directories
644	<code>rw-r--r--</code>	Regular files (default for new files)
700	<code>rw-----</code>	Sensitive files, private directories
600	<code>rw-----</code>	SSH keys, passwords, secrets
777	<code>rw-rw-rw-x</code>	Don't do this (it's a security nightmare)

Why Files vs. Directories is Confusing (And Critical)

Here's where people mess up:

➤ Scenario: You create a script

```
touch myscript.sh  
ls -l myscript.sh  
-rw-r--r-- 1 you you 0 Nov 24 10:30 myscript.sh
```

You write code in it, then try to run it:

```
./myscript.sh  
bash: permission denied: ./myscript.sh
```

Why? The x (execute) bit isn't set. The file exists, you own it, but you can't run it.

Fix:

```
chmod +x myscript.sh  
./myscript.sh # Now it works
```



Linux Fundamentals



Scenario: You want a private directory

```
mkdir /home/hosni/secrets  
chmod 600 /home/hosni/secrets  
cd /home/hosni/secrets  
bash: cd: /home/hosni/secrets: Permission denied
```

Why? For directories, x means "you can enter it." You set rw (read + write) but forgot x (execute).

Fix:

```
chmod 700 /home/hosni/secrets # rwx-----  
cd /home/hosni/secrets # Now it works
```

➤ The core difference:

- **Files:** x = "can I run this?"
- **Directories:** x = "can I enter and access files inside?"



The Owner: Group Model (Why It's Genius)

Most people think permissions are just about privacy. They're about collaboration.

➤ Real example: Web server

```
-rw-r----- 1 ali www-data 2048 Nov 24 /var/www/config.php
```

What this means:

- Ali (developer) can read and write the file
- www-data group (the web server) can read it
- No one else can access it

Ali can update the app without using sudo. The web server can read the config. Other users can't touch it. Perfect.

Why this matters: You don't need to make everything world-readable or executable. You give specific groups specific access. Security through precision, not paranoia.

Setting the group

```
chown ali:developers myfile.txt # Change owner and group
chgrp developers myfile.txt      # Just change the group
chmod g+w myfile.txt            # Give the group write access
```

Now anyone in the developers group can collaborate without needing root access.

Real Examples: When Bad Permissions Cause Nightmares

➤ The chmod 777 Disaster

```
chmod 777 /var/www/uploads
```

Now literally anyone can read, write, and execute everything in that directory. An attacker could:

- Upload malicious scripts
- Overwrite legitimate files
- Delete everything

Better:

```
chmod 755 /var/www/uploads  
# Only owner can write, everyone can read
```

Or if you want a specific user to upload:

```
chown www-data:www-data /var/www/uploads  
chmod 750 /var/www/uploads  
# Owner (www-data) full access, group can read/enter,  
others blocked
```

➤ SSH Keys Are World-Readable

```
ls -l ~/.ssh/id_rsa  
-rw-rw-r-- 1 you you 1876 Nov 24 id_rsa
```

This is catastrophic. Your private SSH key is readable by everyone on the system. An attacker could:

- Log in as you to other servers
- Impersonate you
- Steal access to your git repos

SSH will actually refuse to use keys with bad permissions:

```
ssh -i ~/.ssh/id_rsa git@github.com  
Permissions 0644 for '/home/you/.ssh/id_rsa' are too open.
```

Fix:

```
chmod 600 ~/.ssh/id_rsa
```

Now only you can read it. This is the default for good reason.

➤ Database Credentials Readable by Everyone

```
ls -l /etc/db_password  
-rw-r--r-- 1 root root 32 Nov 24 db_password
```

Every user on the system can read your database password. Game over.

Better:

```
chmod 600 /etc/db_password      # Only root can read  
chown root:root /etc/db_password
```

Or if your app needs to read it:

```
chmod 640 /etc/db_password  
# Owner and group can read, no one else  
chown root:myapp /etc/db_password
```

Each file has exactly the permissions it needs. Nothing more, nothing less.



The chmod Command Patterns You'll Use 90% of the Time

Make a script executable

```
chmod +x script.sh
```

Make a file readable by your group but not the world

```
chmod 640 config.txt
```

Make a directory traversable but not readable

```
chmod 711 /home/user/private
```

Make a directory fully private

```
chmod 700 /home/user/secrets
```

Fix SSH key permissions (this is critical)

```
chmod 600 ~/.ssh/id_rsa
```

Make a file world-readable but not writable

```
chmod 644 readme.txt
```

Give group write access to a directory

```
chmod g+w /project/shared
```

Why This Foundation Matters

Permissions aren't optional. They're the foundation of Linux security.

- **You control what gets exposed** — not by accident, but by design
- **You enable collaboration** — teams can work without root access
- **You prevent catastrophic mistakes** — bad permissions catch many vulnerabilities
- **You're ready for DevOps** — containers, cloud, automation all rely on this
- **You actually understand your system** — instead of just hoping it's secure

One `chmod 777` on the wrong directory and your whole system is vulnerable. One `chmod 600` on your SSH key and you've blocked attackers. Permissions are your responsibility.



Linux Fundamentals



Common Mistakes to Avoid

- Using `chmod 777` to "fix" permission errors (you're fixing the symptom, not the cause)
- Forgetting that files need `x` to be executable and directories need `x` to be entered
- Ignoring group permissions (they're powerful for collaboration)
- Making SSH keys world-readable (one of the most common security fails)
- Setting bad permissions and forgetting about them (audit your important files)

